REVISION CHEAT SHEET:

SECURITY / PROD

ASP.NET Core Security, Integration and Production Backend Design

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

- Why This Topic Matters
- **SECURITY MENTAL MODEL**
- Security Is a System Property
- **Trust Boundaries**
- **AUTHENTICATION**
- What Is Authentication?
- **Authentication Pipeline**
- **Authentication Schemes**
- **Middleware Registration**
- **JWT Bearer Authentication**
- **What a JWT Is Not**
- **Claims**
- **Cookie Authentication**
- **Authentication Does Not Secure Endpoints by Itself**
- **AUTHORIZATION**
- What Is Authorization?
- **Simple Authorization**
- **Secure-by-Default Fallback Policy**
- **Role-Based Authorization**
- **Claims-Based Authorization**
- **Policy-Based Authorization**
- **Custom Authorization Requirement**
- **Resource-Based Authorization**
- **MULTI-TENANT SECURITY**
- Tenant Isolation Is a Security Requirement
- **Do Not Trust TenantId from Request Body**
- **Tenant Context Service**
- **Tenant-Aware Data Access**
- **Defense in Depth for Tenant Isolation**
- **CORS**
- What Is CORS?
- **CORS Does Not Protect the API from Non-Browser Clients**
- **Configure Specific Origins**
- **Avoid AllowAnyOrigin in Sensitive APIs**
- **Credentials and CORS**
- **CSRF / XSRF**
- What Is CSRF?
- **When CSRF Is Most Relevant**
- **Bearer Tokens and CSRF**
- **Antiforgery**
- **HTTPS AND TRANSPORT SECURITY**
- Use HTTPS in Production
- **HTTPS Redirection**
- **HSTS**
- **REVERSE PROXIES AND FORWARDED HEADERS**
- Why Forwarded Headers Matter
- **Common Forwarded Headers**
- **Do Not Trust Forwarded Headers from Everyone**
- **SECRET MANAGEMENT**
- What Is a Secret?
- **Development User Secrets**
- **Production Secrets**
- **Environment Variables Are Not Automatically Secret**
- **Secret Rotation**
- **RATE LIMITING**
- Why Rate Limiting Exists
- **Example Fixed-Window Policy**
- **Rate-Limit by User or Tenant**
- **Concurrency Limiter**
- **Rate Limiting Is Not Complete DDoS Protection**
- **INPUT AND DATA SECURITY**

Never Trust Request Input
Boundary Validation
Domain Validation Still Matters
Over-Posting
SQL Injection
Dangerous Dynamic SQL
SAFE ERROR HANDLING AND LOGGING
Client Errors vs Server Diagnostics
Log Structured Data
Do Not Log Sensitive Data
Correlation and Tracing
EXTERNAL HTTP INTEGRATIONS
External Services Fail
Use IHttpClientFactory
Typed Client
Set Timeouts
RESILIENCE
What Is Resilience?
Standard HTTP Resilience Handler
Retry
Retry Safe Operations
Circuit Breaker
Timeout
Bulkhead / Concurrency Isolation
MESSAGING AND BACKGROUND INTEGRATION
Synchronous vs Asynchronous Integration
Example: Receipt Delivery
OUTBOX PATTERN
The Dual-Write Problem
Transactional Outbox
Outbox Entity
Outbox Processing Is Usually At-Least-Once
IDEMPOTENT MESSAGE CONSUMERS
Consumer Deduplication
Integration Events Need Stable Contracts
Event Versioning
BACKGROUND SERVICES
BackgroundService
Propagate Shutdown Cancellation
HEALTH CHECKS
Why Health Checks Exist
Liveness vs Readiness
Why Database Should Not Always Be Liveness
Example Health Endpoints
Health Checks Are Not Monit SECURITY / PROD
OBSERVABILITY
Three Major Signals
Logs
Metrics
Traces
Useful Fundraising Metrics
Alert on Symptoms That Matter
PRODUCTION DATABASE SECURITY
Least Privilege
Application vs Migration Credentials
Backups and Recovery Are Part of Backend Design
DEPLOYMENT AND SCALE
Stateless API Instances
Sticky Sessions Are Not a Substitute for Good Architecture
Distributed Cache
Cache Tenant-Aware Keys
DATA PROTECTION AND SERVER FARMS
Shared Cryptographic State
DEPLOYMENT-SAFE DATABASE CHANGES
Avoid Breaking Rolling Deployments
Don't Let Every API Instance Run Dangerous Migrations
GRACEFUL SHUTDOWN
Why Graceful Shutdown Matters
Cancellation Propagation
Do Not Swallow OperationCanceledException Blindly
SECURITY HEADERS
Useful HTTP Security Headers
PRODUCTION PROGRAM.CS
Example Production-Oriented Registration
Example Production Pipeline
COMPLETE FUNDRAISING SECURITY FLOW
Create Gift Request
Request Processing
Application Flow
Transactional Integration
Background Flow
Diagram 1 — Production Fundraising Backend
FAILURE DESIGN
SQL Server Is Down
Payment Provider Is Slow

Message Broker Is Down
Duplicate Message Arrives
API Instance Crashes
PERFORMANCE AND SECURITY TOGETHER
Many Security Controls Also Protect Performance
Avoid Expensive Authorization N+1
Avoid Security Through Obscurity
SECURITY TESTING
Integration Tests Should Include Security
Test Tenant Isolation
Test Idempotency
Test Rate Limiting
Test Integration Failure
COMMON PRODUCTION MISTAKES
Mistake 1 — Authentication Without Authorization
Mistake 2 — Trusting TenantId from Client
Mistake 3 — CORS as Security
Mistake 4 — AllowAnyOrigin + Credentials
Mistake 5 — Secrets in appsettings.json
Mistake 6 — User Secrets in Production
Mistake 7 — New HttpClient Per Request
Mistake 8 — Retry Every Failure
Mistake 9 — No Timeouts
Mistake 10 — Database + Message Broker Dual Write
Mistake 11 — No Consumer Idempotency
Mistake 12 — Liveness Depends on Every External System
Mistake 13 — Logging Tokens and Secrets
Mistake 14 — Unbounded Queries
Mistake 15 — Shared Singleton Tenant State
Mistake 16 — Trusting X-Forwarded-For from Everyone
Mistake 17 — Auto-Migrating From Every Instance
Mistake 18 — Exposing Detailed Health Checks Publicly
Mistake 19 — Returning Stack Traces
Mistake 20 — No Graceful Shutdown
BEST PRACTICES
12 PRACTICAL EXERCISES
Exercise 1 — Authentication
Exercise 2 — Fallback Authorization
Exercise 3 — Permission Policy
Exercise 4 — Tenant Isolation
Exercise 5 — CORS
Exercise 6 — Secrets
Exercise 7 — Rate Limiting
Exercise 8 — SQL Injection Review
Exercise 9 — Resilient HttpClient
Exercise 10 — Transactional Outbox
Exercise 11 — Idempotent Consumer
Exercise 12 — Production Health
EXACTLY 25 INTERVIEW QUESTIONS
QUICK REVISION
FINAL ROADMAP CONCEPT FLOW
Final Fundraising Platform Mental Model
ROADMAP COMPLETE — 40 / 40

SECURITY / PROD

Why This Topic Matters

A backend application is not production-ready merely because:

EXAMPLE / FLOW

```
GET /api/gifts
```

works correctly.

Production systems must also answer:

EXAMPLE / FLOW

```
Who is calling the API?

How is the caller authenticated?

What is the caller allowed to do?

Which Tenant's data can they access?

Can one client overwhelm the API?

What happens when SQL Server is unavailable?
```

```
What happens when a payment service times out?

Should the request be retried?

Could retrying create duplicate Gifts?

How are secrets stored?

How are failures logged without leaking private information?

Can the application scale to multiple instances?

What happens while a deployment is shutting an instance down?

How do operators know whether the application is healthy?
```

SECURITY / PROD

This topic therefore combines three major concerns:

EXAMPLE / FLOW

```
SECURITY
+
INTEGRATION
+
PRODUCTION OPERATIONS
```

For the Fundraising Platform:

EXAMPLE / FLOW

```
Angular / External Client
        ↓
HTTPS
        ↓
ASP.NET Core API
        ↓
Authentication
        ↓
Authorization
        ↓
Tenant Isolation
        ↓
Application Layer
        ↓
Domain
        ↓
EF Core / SQL Server
        ↓
External Integrations
        ↓
Background Processing
        ↓
Monitoring / Operations
```

SECURITY MENTAL MODEL

Security Is a System Property

Security should not exist only in:

EXAMPLE / FLOW

```
LoginController
```

or:

EXAMPLE / FLOW

SECURITY / PROD

```
[Authorize]
```

A secure backend applies controls at several boundaries:

EXAMPLE / FLOW

```
NETWORK
    ↓
HTTPS

HTTP PIPELINE
    ↓
Authentication
Authorization
CORS
Rate limiting

APPLICATION
    ↓
Use-case authorization
Tenant isolation
Input validation

DOMAIN
    ↓
Business invariants

DATA
    ↓
Tenant filters
Database permissions
Constraints
Parameterized SQL

INTEGRATIONS
    ↓
Credentials
Timeouts
TLS
Signed/validated messages

OPERATIONS
    ↓
Secret storage
Logging
Monitoring
Patch/update strategy
```

This is **defense in depth**.

One control should not be your only protection.

Trust Boundaries

A trust boundary is a point where data crosses from one trust level to another.

Examples:

EXAMPLE / FLOW

```
Browser
    ↓
API
```

SECURITY / PROD

EXAMPLE / FLOW

```
API
    ↓
SQL Server
```

EXAMPLE / FLOW

```
API
    ↓
Payment provider
```

EXAMPLE / FLOW

```
Message broker
    ↓
Background consumer
```

Every boundary should ask:

EXAMPLE / FLOW

```
Who sent this?

Can I trust the identity?

Is this data valid?

Is the caller authorized?

Can this input cause excessive resource use?

Could this request be duplicated?

Can this response expose sensitive information?
```

Never assume:

EXAMPLE / FLOW

```
"It came from our Angular application,
therefore it is safe."
```

Clients can be modified or replaced entirely.

The server must enforce its own rules.

AUTHENTICATION

What Is Authentication?

Authentication answers:

> **Who is the caller?**

ASP.NET Core authentication creates an authenticated identity, normally represented by:

```C#
ClaimsPrincipal
```

available through:

```C#
HttpContext.User
```

Authentication and authorization are separate. Authentication establishes identity; authorization determines what that identity can do. ASP.NET Core authentication is implemented through registered authentication schemes and handlers.

Authentication Pipeline

Conceptually:

```EXAMPLE / FLOW
HTTP Request
    ↓
Authentication Middleware
    ↓
Authentication Scheme
    ↓
Authentication Handler
    ↓
Validate credential/token/cookie
    ↓
ClaimsPrincipal
    ↓
HttpContext.User
```

Then:

```EXAMPLE / FLOW
Authorization
```

can evaluate the authenticated identity.

Authentication Schemes

ASP.NET Core supports authentication schemes.

Common examples include:

```EXAMPLE / FLOW
JWT Bearer

Cookies

OpenID Connect

OAuth-related handlers

Certificate authentication

Windows authentication
```

A scheme identifies which handler should process authentication behavior.

For example:

C#

SECURITY / PROD

```
builder.Services
    .AddAuthentication(
        JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(
        options =>
        {
            options.Authority = builder.Configuration["Authentication:Authority"];

            options.Audience = "fundraising-api";
        });
```

Microsoft's current authentication guidance describes `AddJwtBearer`, `AddCookie`, and other scheme-specific handlers as registrations attached to `AddAuthentication`.

Middleware Registration

Registering authentication services:

C#

```
builder.Services
    .AddAuthentication(...)
    .AddJwtBearer(...);
```

does not itself execute authentication for each request.

The request pipeline uses:

C#

```
app.UseAuthentication();
```

and it must run before middleware that expects an authenticated user.

Typical ordering:

EXAMPLE / FLOW

```
Routing
    ↓
Authentication
    ↓
Authorization
    ↓
Endpoint
```

JWT Bearer Authentication

For an API used by Angular or service clients, JWT bearer authentication is common.

Client sends:

EXAMPLE / FLOW

```
Authorization: Bearer eyJ...
```

API validates things such as:

EXAMPLE / FLOW

```
Signature

Issuer
```

```
Audience
*
Expiration

Other configured token rules
```

SECURITY / PROD

and builds:

```
C#
ClaimsPrincipal
```

Do not write your own JWT validation logic unless there is an exceptional reason.

Use established authentication middleware and a trusted identity provider.

What a JWT Is Not

A JWT should not be treated as:

EXAMPLE / FLOW

```
encrypted secret storage
```

JWT claims are commonly encoded and signed, not necessarily encrypted.

Do not put information into access-token claims simply because:

EXAMPLE / FLOW

```
"users cannot see it."
```

A client holding the token can generally inspect its claims.

Never place:

EXAMPLE / FLOW

```
Passwords

Database credentials

Private keys

Sensitive secret values
```

inside ordinary access-token claims.

Claims

A claim describes something about the authenticated identity.

Examples:

EXAMPLE / FLOW

```
sub
    →
user identifier

tenant_id
    →
tenant context

role
    →
user role
```

```
permission
    →
fine-grained permission
```

SECURITY / PROD

Code:

C#

```
string? userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
```

Claims become inputs into authorization decisions.

Do not confuse:

EXAMPLE / FLOW

```
claim exists
```

with:

EXAMPLE / FLOW

```
claim must automatically be trusted
```

The identity/token itself must first be validated by the configured authentication system.

Cookie Authentication

Cookie authentication is common for browser-oriented applications.

After authentication:

EXAMPLE / FLOW

```
Server
    ↓
sets authentication cookie

Browser
    ↓
automatically sends cookie
```

This automatic browser behavior has important CSRF implications that we'll discuss later.

For an Angular/API architecture, authentication design depends on the identity architecture. Avoid choosing cookie vs bearer simply because one example project used it.

Authentication Does Not Secure Endpoints by Itself

This is critical.

You can configure:

C#

```
builder.Services
    .AddAuthentication(...)
```

and still have publicly accessible endpoints.

Microsoft explicitly states that configuring authentication does not automatically require authentication for endpoints. Authorization metadata or a fallback authorization policy is still required.

AUTHORIZATION

SECURITY / PROD

What Is Authorization?

Authorization answers:

> **Is this authenticated caller allowed to perform this operation?**

For example:

EXAMPLE / FLOW

```
User Alice authenticated successfully.
```

That does not automatically mean:

EXAMPLE / FLOW

```
Alice may refund Gift 100.
```

Authentication:

EXAMPLE / FLOW

```
Who are you?
```

Authorization:

EXAMPLE / FLOW

```
What may you do?
```

ASP.NET Core supports declarative role-based authorization and a richer policy-based model.

Simple Authorization

Controller:

C#

```
[Authorize]
[ApiController]
[Route("api/gifts")]
public sealed class GiftsController :
    ControllerBase
{
}
```

This requires the default authorization policy.

Individual public action:

C#

```
[AllowAnonymous]
[HttpGet("public-summary")]
public IActionResult PublicSummary()
{
    ...
}
```

Secure-by-Default Fallback Policy

SECURITY / PROD

For systems where nearly every endpoint should require authentication, a strong design is:

```C#
builder.Services
    .AddAuthorization(
        options =>
        {
            options.FallbackPolicy =
                new AuthorizationPolicyBuilder()
                    .RequireAuthenticatedUser()
                    .Build();
        });
```

Then endpoints are protected unless explicitly marked:

```C#
[AllowAnonymous]
```

Microsoft documents fallback policies specifically for requiring authenticated users by default.

This helps prevent accidentally creating a new public endpoint.

Role-Based Authorization

Example:

```C#
[Authorize(
    Roles =
        "FundraisingAdministrator")]
```

Roles are useful when permissions align well with stable organizational roles.

Examples:

EXAMPLE / FLOW

```
Administrator

GiftProcessor

ReadOnlyUser
```

But large applications often outgrow simple role checks.

You may eventually have:

EXAMPLE / FLOW

```
Administrator

GiftProcessor

GiftRefundProcessor

CampaignManager

FundManager

Auditor
```

```
ReportingUser
```

SECURITY / PROD

and many combinations.

Policy-based authorization provides more flexibility.

Claims-Based Authorization

Example policy:

C#

```
builder.Services
    .AddAuthorization(
        options =>
        {
            options.AddPolicy(
                "CanRefundGift",
                policy =>
                    policy.RequireClaim(
                        "permission",
                        "gifts.refund"));
        });
```

Controller:

C#

```
[Authorize(
    Policy =
        "CanRefundGift")]
[HttpPost(
    "{giftId:long}/refund")]
public async Task<IActionResult>
    Refund(...)
{
}
```

Now authorization is based on:

EXAMPLE / FLOW

```
permission = gifts.refund
```

rather than one hard-coded role name.

Policy-Based Authorization

An ASP.NET Core policy contains one or more requirements.

Conceptually:

EXAMPLE / FLOW

```
CanRefundGift Policy
        ↓
Authenticated user
        +
PermissionRequirement("gifts.refund")
        +
possibly additional rules
```

Handlers evaluate requirements.

Microsoft's current policy-based authorization documentation defines a policy as a collection of authorization requirements evaluated to decide whether access should be granted.

SECURITY / PROD

Custom Authorization Requirement

Example:

C#

```
public sealed record
    MinimumGiftApprovalRequirement(
        decimal MaximumAmount)
    :
        IAuthorizationRequirement;
```

Handler:

C#

```
public sealed class
    MinimumGiftApprovalHandler
    :
        AuthorizationHandler<
            MinimumGiftApprovalRequirement>
{
    protected override Task
        HandleRequirementAsync(
            AuthorizationHandlerContext context,
            MinimumGiftApprovalRequirement requirement)
    {
        if (
            context.User.HasClaim(
                "permission",
                "gifts.approve"))
        {
            context.Succeed(
                requirement);
        }

        return Task.CompletedTask;
    }
}
```

This can encode more sophisticated permission logic.

Resource-Based Authorization

Sometimes authorization depends on the actual resource.

Example:

EXAMPLE / FLOW

```
User may edit Gift
only when Gift belongs to user's Tenant.
```

or:

EXAMPLE / FLOW

```
User may approve Gift
only if assigned to that Campaign.
```

You cannot always solve that with:

C#

```
[Authorize]
```

SECURITY / PROD

because attributes run before the action loads the resource.

ASP.NET Core supports imperative resource-based authorization through:

C#

```
IAuthorizationService
```

and `AuthorizeAsync`. Microsoft specifically notes that resource-based authorization often requires loading the resource and then calling the authorization service.

Example:

C#

```
Gift? gift = await giftRepository.FindAsync(tenantId, giftId, cancellationToken);

if (gift is null)
{
    return NotFound();
}

AuthorizationResult result =
    await authorizationService
        .AuthorizeAsync(
            User,
            gift,
            "CanManageGift");

if (!result.Succeeded)
{
    return Forbid();
}
```

MULTI-TENANT SECURITY

Tenant Isolation Is a Security Requirement

Your Fundraising Platform includes:

EXAMPLE / FLOW

```
Tenant
    ↓
Constituent

Tenant
    ↓
Campaign

Tenant
    ↓
Fund

Tenant
    ↓
Gift
```

A user from Tenant A must not retrieve:

EXAMPLE / FLOW

```
Tenant B Gift
```

SECURITY / PROD

even if they know its ID.

This is one of the most important backend security boundaries.

Do Not Trust TenantId from Request Body

Dangerous:

C#

```
{
  "tenantId": 999,
  "constituentId": 100,
  "amount": 500
}
```

followed by:

C#

```
Gift.Create(
    request.TenantId,
    ...);
```

A malicious client can simply change:

EXAMPLE / FLOW

```
999
```

to another Tenant ID.

Instead, obtain Tenant context from a trusted server-side identity/authorization process.

Conceptually:

EXAMPLE / FLOW

```
Validated identity
    ↓
Determine allowed Tenant context
    ↓
Application use case
```

Tenant Context Service

Example:

C#

```
public interface ICurrentTenant
{
    long TenantId
    {
        get;
    }
}
```

Scoped implementation:

C#

SECURITY / PROD

```
public sealed class CurrentTenant(
    IHttpContextAccessor accessor)
    :
        ICurrentTenant
{
    public long TenantId
    {
        get
        {
            string value =
                accessor.HttpContext?
                    .User
                    .FindFirstValue(
                        "tenant_id")
                ??
                throw new
                    UnauthorizedAccessException();

            return long.Parse(
                value,
                CultureInfo.InvariantCulture);
        }
    }
}
```

In a real multi-tenant design, you may also need to verify:

EXAMPLE / FLOW

```
user actually belongs to Tenant

Tenant is active

requested Tenant context is allowed
```

rather than simply trusting an arbitrary claim name.

Tenant-Aware Data Access

Repository:

C#

```
public Task<Gift?>
    FindAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken)
{
    return _dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.TenantId
                    ==
                    tenantId
                &&
                gift.GiftId
                    == giftId, cancellationToken);
}
```

Not:

C#

```
return _dbContext.Gifts
    .SingleOrDefaultAsync(
        gift =>
            gift.GiftId
                == giftId);
```

SECURITY / PROD

Tenant isolation belongs in data-access design, not just controllers.

Defense in Depth for Tenant Isolation

A stronger model combines:

EXAMPLE / FLOW

```
Authenticated Tenant identity

Authorization policies

Tenant-aware repositories

EF global query filters where appropriate

Database constraints

Separate schemas/databases where architecture requires them

Integration tests
```

Do not rely solely on:

EXAMPLE / FLOW

```
developer remembers to add
.Where(x => x.TenantId == tenantId)
```

to every query.

CORS

What Is CORS?

CORS means:

> Cross-Origin Resource Sharing.

Browsers enforce a same-origin policy.

For example:

EXAMPLE / FLOW

```
Angular:
https://app.example.com
```

calling:

EXAMPLE / FLOW

```
API:
https://api.example.com
```

is cross-origin because the hosts differ.

CORS allows the server to tell the browser which cross-origin requests may be exposed to frontend JavaScript.

SECURITY / PROD

Microsoft emphasizes that CORS **relaxes** the browser's same-origin policy; it is not authentication or an API security boundary.

CORS Does Not Protect the API from Non-Browser Clients

A malicious script using:

EXAMPLE / FLOW

```
curl

Postman

custom C# client

Python script
```

does not care about browser CORS enforcement.

Therefore:

EXAMPLE / FLOW

```
CORS
≠
Authentication

CORS
≠
Authorization

CORS
≠
Tenant Security
```

Your API must still authenticate and authorize every protected operation.

Configure Specific Origins

Example:

C#

```
builder.Services
    .AddCors(
        options =>
        {
            options.AddPolicy(
                "Angular",
                policy =>
                {
                    policy
                        .WithOrigins(
                            "https://app.example.com")
                        .AllowAnyHeader()
                        .AllowAnyMethod();
                });
        });
```

Pipeline:

C#

```
app.UseRouting();

app.UseCors(
    "Angular");

app.UseAuthentication();

app.UseAuthorization();
```

SECURITY / PROD

Microsoft documents `UseCors` after routing and before authorization in endpoint-routing configurations.

Avoid AllowAnyOrigin in Sensitive APIs

This:

C#

```
.AllowAnyOrigin()
```

allows any browser origin to issue CORS requests.

Microsoft explicitly warns that `AllowAnyOrigin` is insecure for sensitive scenarios, and combining arbitrary origins with credentials is unsafe.

Prefer an allow-list:

C#

```
.WithOrigins(
    "https://app.example.com")
```

Credentials and CORS

Cookie-based cross-origin requests may require:

C#

```
.AllowCredentials()
```

but then the allowed origins must be explicitly restricted.

This combination is dangerous:

C#

```
.AllowAnyOrigin()
.AllowCredentials()
```

ASP.NET Core prevents this invalid/insecure configuration.

CSRF / XSRF

What Is CSRF?

Cross-Site Request Forgery occurs when a malicious site causes a victim's browser to submit a state-changing request to another site using credentials the browser automatically attaches.

Classic example:

EXAMPLE / FLOW

```
User logged into fundraising.example.com
        ↓
Authentication cookie stored
```

```
        ↓
User visits malicious.example.com
        ↓
Malicious page submits POST
        ↓
Browser automatically sends fundraising cookie
```

SECURITY / PROD

If the server accepts the request without appropriate protection:

EXAMPLE / FLOW

```
attacker may perform action as victim
```

When CSRF Is Most Relevant

CSRF is especially relevant when authentication credentials are automatically sent by the browser, such as:

EXAMPLE / FLOW

```
Cookies

Windows authentication
```

Microsoft's antiforgery documentation specifically treats automatically sent authentication context as a CSRF threat.

Bearer Tokens and CSRF

A bearer token explicitly attached using:

EXAMPLE / FLOW

```
Authorization: Bearer ...
```

is not normally automatically attached by the browser to arbitrary cross-site form requests.

Therefore traditional cookie-style CSRF is reduced in that architecture.

However:

EXAMPLE / FLOW

```
Bearer tokens
```

introduce other risks, especially:

EXAMPLE / FLOW

```
XSS

token theft

unsafe token storage
```

So:

EXAMPLE / FLOW

```
"No CSRF"
```

does not mean:

EXAMPLE / FLOW

```
"No browser security problems."
```

SECURITY / PROD

Antiforgery

ASP.NET Core provides antiforgery services for appropriate browser/form scenarios.

Microsoft's guidance recommends antiforgery defenses for cookie-based browser interactions and notes that ASP.NET Core's antiforgery implementation relies on Data Protection.

For a cookie-authenticated Angular architecture, the browser/API CSRF design must be deliberate.

HTTPS AND TRANSPORT SECURITY

Use HTTPS in Production

HTTP without TLS can expose:

EXAMPLE / FLOW

```
Authorization headers

Cookies

Request bodies

Personal data

Gift information

Tokens
```

in transit.

Production APIs should use:

EXAMPLE / FLOW

```
HTTPS/TLS
```

end to end at the appropriate trust boundaries.

Microsoft recommends HTTPS for ASP.NET Core production applications. For API projects, it specifically notes that simply redirecting an already-sensitive HTTP request cannot undo the fact that the initial request was sent insecurely.

HTTPS Redirection

Typical web application:

C#

```
app.UseHttpsRedirection();
```

But pure APIs often sit behind:

EXAMPLE / FLOW

```
load balancer
reverse proxy
API gateway
```

that already enforces HTTPS.

Avoid redundant/misconfigured redirects that break:

EXAMPLE / FLOW

SECURITY / PROD

```
CORS preflight

proxy routing
```

Microsoft notes that proxy infrastructure can own TLS redirection/HSTS instead of the ASP.NET Core process.

HSTS

HSTS tells compatible browsers to prefer/require HTTPS for future requests.

Example:

C#

```
if (!app.Environment.IsDevelopment())
{
    app.UseHsts();
}
```

However, HSTS is mainly a browser mechanism. Microsoft notes that API clients such as native mobile/desktop clients may not honor it.

REVERSE PROXIES AND FORWARDED HEADERS

Why Forwarded Headers Matter

Production topology may look like:

EXAMPLE / FLOW

```
Internet
   ↓
Load Balancer
   ↓ TLS terminates here
Reverse Proxy
   ↓ HTTP internally
Kestrel
```

Kestrel may see:

EXAMPLE / FLOW

```
Scheme = http
Remote IP = proxy
```

even though the original request was:

EXAMPLE / FLOW

```
HTTPS
from real client IP
```

Forwarded headers can restore the original information.

Common Forwarded Headers

Typical headers include:

EXAMPLE / FLOW

```
X-Forwarded-For
    →
original client address

X-Forwarded-Proto
    →
original scheme

X-Forwarded-Host
    →
original host
```

SECURITY / PROD

ASP.NET Core's Forwarded Headers Middleware can update:

C#

```
HttpContext.Connection.RemoteIpAddress

HttpContext.Request.Scheme

HttpContext.Request.Host
```

based on trusted proxy information.

Do Not Trust Forwarded Headers from Everyone

A client can fabricate:

EXAMPLE / FLOW

```
X-Forwarded-For: 127.0.0.1
```

unless your infrastructure restricts which proxies are trusted.

Therefore configure:

EXAMPLE / FLOW

```
KnownProxies

KnownNetworks
```

or appropriate trusted infrastructure.

Microsoft explicitly calls out spoofing risks around forwarded headers.

SECRET MANAGEMENT

What Is a Secret?

Examples:

EXAMPLE / FLOW

```
Database password

API key

OAuth client secret

Signing credential

Private key
```

```
Payment provider secret
```

SECURITY / PROD

Do not store these directly in:

EXAMPLE / FLOW

```
source code

Git repository

public configuration

logs
```

Microsoft's security guidance explicitly says passwords and sensitive values should not be stored in source code or configuration files, and production secrets should be accessed through controlled secret-management mechanisms.

Development User Secrets

For development:

C#

```
dotnet user-secrets set "Payment:ApiKey" "..."
```

keeps the secret outside the project directory.

But Secret Manager:

EXAMPLE / FLOW

```
is development-only

does not encrypt secrets

is not a trusted production vault
```

Microsoft explicitly warns that user secrets are not encrypted and should only be used for development.

Production Secrets

In production, use an appropriate controlled secret source such as:

EXAMPLE / FLOW

```
Cloud secret vault

Managed identity

Platform secret store

Protected deployment configuration
```

Example on Azure:

EXAMPLE / FLOW

```
Azure Key Vault
```

Microsoft specifically recommends controlled production secret access such as Key Vault rather than deploying secrets with the application.

Environment Variables Are Not Automatically Secret

SECURITY / PROD

Environment variables are useful for configuration but can often be inspected by:

EXAMPLE / FLOW

```
process administrators

host tooling

compromised processes
```

Microsoft notes that environment variables are commonly stored as unencrypted text and may require additional protection.

So:

EXAMPLE / FLOW

```
Environment variable
```

does not automatically equal:

EXAMPLE / FLOW

```
secure vault
```

Secret Rotation

Production design should consider:

EXAMPLE / FLOW

```
How is secret changed?

Does app restart?

Can old/new secret overlap?

How are credentials revoked?

How quickly does application reload?
```

A secret that can never be safely rotated is an operational risk.

RATE LIMITING

Why Rate Limiting Exists

Rate limiting can:

EXAMPLE / FLOW

```
reduce abuse

protect expensive endpoints

provide fair usage

control concurrency

protect downstream resources
```

ASP.NET Core includes rate-limiting middleware with fixed-window, sliding-window, token-bucket, and concurrency strategies.

SECURITY / PROD

Example Fixed-Window Policy

C#

```
builder.Services
    .AddRateLimiter(
        options =>
        {
            options.AddFixedWindowLimiter(
                "GiftWrites",
                limiter =>
                {
                    limiter.PermitLimit = 30;

                    limiter.Window = TimeSpan.FromMinutes(1);

                    limiter.QueueLimit = 0;
                });
        });
```

Pipeline:

C#

```
app.UseRateLimiter();
```

Endpoint:

C#

```
[EnableRateLimiting(
    "GiftWrites")]
[HttpPost]
public ...
```

Rate-Limit by User or Tenant

A single global limit such as:

EXAMPLE / FLOW

```
100 requests/minute total
```

may let one user consume the allowance for everyone.

Partition by:

EXAMPLE / FLOW

```
Tenant

User ID

API key

client identity
```

where appropriate.

Microsoft's rate-limiting guidance explicitly discusses partitioning traffic by identifiers such as user, IP, or API key to improve fairness and control.

Concurrency Limiter

SECURITY / PROD

A concurrency limiter controls:

```
EXAMPLE / FLOW
How many requests may execute simultaneously?
```

This can protect expensive endpoints such as:

```
EXAMPLE / FLOW
PDF generation

large exports

CPU-heavy reporting
```

It differs from:

```
EXAMPLE / FLOW
requests per minute
```

because it focuses on simultaneous work.

Rate Limiting Is Not Complete DDoS Protection

Application rate limiting happens:

```
EXAMPLE / FLOW
after traffic reaches your infrastructure
```

Large-scale denial-of-service protection often also belongs at:

```
EXAMPLE / FLOW
CDN

WAF

API gateway

load balancer

cloud network edge
```

Use application rate limiting as one layer.

INPUT AND DATA SECURITY

Never Trust Request Input

All request values are untrusted:

```
EXAMPLE / FLOW
Route values

Query strings

JSON

Headers
```

```
Cookies

Uploaded files
```

SECURITY / PROD

Even if:

EXAMPLE / FLOW

```
Angular normally sends valid values
```

an attacker can send custom HTTP requests.

Boundary Validation

Validate:

EXAMPLE / FLOW

```
Required fields

Lengths

Ranges

Allowed values

Pagination limits

File sizes/types

Request format
```

Example:

C#

```
public sealed record GiftSearchRequest
{
    [Range(
        1,
        200)]
    public int PageSize
    {
        get;
        init;
    } = 50;
}
```

Domain Validation Still Matters

Boundary validation:

EXAMPLE / FLOW

```
Amount field parses correctly.
```

Domain validation:

EXAMPLE / FLOW

```
Gift cannot be completed until fully allocated.
```

Keep both.

HTTP validation does not replace domain invariants.

SECURITY / PROD

Over-Posting

Avoid:

C#
```
public Task Create(
    Gift entity)
```

where the client can potentially supply:

EXAMPLE / FLOW
```
TenantId

Status

Audit values

internal IDs

IsDeleted
```

Use dedicated DTOs:

C#
```
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal Amount);
```

Then server supplies:

EXAMPLE / FLOW
```
Tenant

Initial status

CreatedAt

internal metadata
```

SQL Injection

With normal EF Core LINQ:

C#
```
dbContext.Gifts
    .Where(
        gift =>
            gift.TenantId
                == tenantId);
```

values are translated to parameters.

Problems arise when dynamically constructing raw SQL using untrusted input.

Use parameterized SQL APIs and allow-list dynamic schema elements.

Microsoft's EF Core guidance distinguishes parameterizing values from dynamically inserting untrusted SQL fragments, noting that identifiers such as column names cannot be parameteri
require careful allow-listing/sanitization.

SECURITY / PROD

Dangerous Dynamic SQL

Bad:

C#
```
string sql =
    $"SELECT * FROM fundraising.Gift WHERE {request.SortColumn} = '{request.Value}'";
```

Now input can affect:

EXAMPLE / FLOW
```
SQL syntax
```

Better:

EXAMPLE / FLOW
```
Map allowed sort options explicitly
```

Example:

C#
```
query =
    request.SortBy switch
    {
        "amount" =>
            query.OrderBy(
                x =>
                    x.TotalAmount),

        "date" =>
            query.OrderBy(
                x =>
                    x.GiftDate),

        _ =>
            query.OrderBy(
                x =>
                    x.GiftId)
    };
```

SAFE ERROR HANDLING AND LOGGING

Client Errors vs Server Diagnostics

Client:

C#
```
{
  "title": "An unexpected error occurred.",
  "status": 500,
  "traceId": "..."
}
```

Server logs:

EXAMPLE / FLOW

```
actual exception
stack trace
dependency failure
trace information
```

SECURITY / PROD

Do not return internal stack traces in production.

Log Structured Data

Prefer:

C#

```
logger.LogInformation(
    "Gift {GiftId} completed for Tenant {TenantId}",
    giftId,
    tenantId);
```

instead of large concatenated strings.

This produces searchable structured properties.

Do Not Log Sensitive Data

Never casually log:

EXAMPLE / FLOW

```
Bearer token

Authentication cookie

Password

Private API key

Full payment-card details

Sensitive donor information
```

Even:

EXAMPLE / FLOW

```
"debug only"
```

logs often reach persistent centralized systems.

Correlation and Tracing

A request may flow:

EXAMPLE / FLOW

```
API
    ↓
Payment Provider
    ↓
SQL Server
    ↓
Message Broker
```

```
    ↓
Receipt Worker
```

SECURITY / PROD

Operational debugging needs identifiers such as:

EXAMPLE / FLOW

```
TraceId

CorrelationId

GiftId

MessageId
```

These allow related operations to be connected.

Do not use:

EXAMPLE / FLOW

```
user's sensitive data
```

as the correlation key.

EXTERNAL HTTP INTEGRATIONS

External Services Fail

Assume every external dependency eventually experiences:

EXAMPLE / FLOW

```
Timeout

DNS issue

Connection failure

5xx response

429 rate limit

Slow response

Partial outage
```

Production code must not assume:

EXAMPLE / FLOW

```
HTTP call always succeeds immediately.
```

Use IHttpClientFactory

Instead of repeatedly doing:

C#

```
using HttpClient client = new();

await client.GetAsync(...);
```

configure an application client:

C#

```
builder.Services
    .AddHttpClient<
        IPaymentGateway,
        PaymentGateway>(
            client =>
            {
                client.BaseAddress = new Uri(builder.Configuration["Payment:BaseUrl"]!);
            });
```

SECURITY / PROD

`IHttpClientFactory` provides centralized client configuration and manages pooled `HttpMessageHandler` lifetimes, helping address connection/DNS lifetime concerns.

Typed Client

C#

```
public sealed class PaymentGateway(
    HttpClient httpClient)
    :
        IPaymentGateway
{
    public async Task<
        PaymentResult>
        ChargeAsync(
            PaymentRequest request,
            CancellationToken cancellationToken)
    {
        using HttpResponseMessage response =
            await httpClient.PostAsJsonAsync(
                "charges",
                request,
                cancellationToken);

        ...
    }
}
```

Application layer depends on:

C#

```
IPaymentGateway
```

Infrastructure owns HTTP details.

Set Timeouts

An external call should not be allowed to wait forever.

Every integration should answer:

EXAMPLE / FLOW

```
How long can this operation reasonably take?
```

Possible timeout levels:

EXAMPLE / FLOW

```
Connection attempt

Individual HTTP attempt
```

```
Overall operation
```

SECURITY / PROD

Timeouts should reflect business expectations.

RESILIENCE

What Is Resilience?

Resilience is the ability to continue functioning appropriately when dependencies experience transient failures.

Modern .NET resilience packages support strategies such as:

EXAMPLE / FLOW

```
Retry

Circuit breaker

Timeout

Rate limiting

Fallback

Hedging
```

Microsoft's current .NET resilience libraries are built on Polly.

Standard HTTP Resilience Handler

Current .NET supports:

C#

```
builder.Services
    .AddHttpClient<
        IPaymentStatusClient,
        PaymentStatusClient>()
    .AddStandardResilienceHandler();
```

Microsoft documents `AddStandardResilienceHandler` as adding a standard resilience pipeline to the `HttpClient`.

The standard pipeline combines strategies including:

EXAMPLE / FLOW

```
rate limiting/concurrency control

total timeout

retry

circuit breaker

per-attempt timeout
```

Configure them according to the external API and your operation semantics.

Retry

Retry is appropriate for some transient failures:

EXAMPLE / FLOW

```
temporary network error

some 5xx responses

429 depending on provider guidance
```

SECURITY / PROD

But retry can be dangerous.

Suppose:

EXAMPLE / FLOW

```
POST payment charge
    ↓
provider processes charge
    ↓
response lost
    ↓
application retries
```

You may charge twice.

Therefore:

> ⓘ Never add retries without understanding whether the operation can safely be repeated.

Retry Safe Operations

Operations such as:

EXAMPLE / FLOW

```
GET payment status
```

are often safer to retry than:

EXAMPLE / FLOW

```
POST create payment
```

For non-idempotent operations, use provider-supported idempotency identifiers or another deduplication strategy before enabling retries.

Circuit Breaker

Suppose payment provider is failing 100% of requests.

Without a circuit breaker:

EXAMPLE / FLOW

```
Every request
    ↓
wait timeout
    ↓
fail
```

A circuit breaker can temporarily stop sending requests after repeated failure.

Conceptually:

EXAMPLE / FLOW

```
CLOSED
    ↓ failures
OPEN
    ↓ wait
HALF-OPEN
    ↓ trial
CLOSED / OPEN
```

SECURITY / PROD

This protects:

EXAMPLE / FLOW

```
your threads/resources

dependency

user response times
```

Timeout

Without timeouts:

EXAMPLE / FLOW

```
Dependency hangs
    ↓
Requests accumulate
    ↓
Memory/connections increase
    ↓
System degrades
```

Timeouts bound the amount of time one dependency is allowed to consume.

Bulkhead / Concurrency Isolation

Suppose payment provider becomes slow.

If all 5,000 incoming requests simultaneously wait on it:

EXAMPLE / FLOW

```
entire API may become unhealthy
```

Concurrency limits isolate dependency consumption.

The same principle applies to:

EXAMPLE / FLOW

```
large exports

slow database workflows

message processing
```

MESSAGING AND BACKGROUND INTEGRATION

Synchronous vs Asynchronous Integration

Synchronous:

EXAMPLE / FLOW

```
API
    ↓ HTTP
Payment Service
    ↓
wait for response
```

SECURITY / PROD

Useful when the response is required before continuing.

Asynchronous:

EXAMPLE / FLOW

```
API
    ↓
Queue / Broker
    ↓
Worker / External System
```

Useful when work:

EXAMPLE / FLOW

```
can happen later

needs durability

should be retried independently

takes a long time
```

Example: Receipt Delivery

Bad synchronous workflow:

EXAMPLE / FLOW

```
Create Gift
    ↓
Save SQL
    ↓
Generate PDF
    ↓
Call email provider
    ↓
wait
    ↓
return HTTP response
```

If email provider is slow:

EXAMPLE / FLOW

```
Gift creation becomes slow.
```

Better architecture:

EXAMPLE / FLOW

```
Create Gift
    ↓
Save Gift + OutboxMessage
    ↓
Commit
    ↓
```

```
Return success
    ↓
Background publisher
    ↓
Receipt worker
    ↓
Email
```

SECURITY / PROD

OUTBOX PATTERN

The Dual-Write Problem

Suppose:

EXAMPLE / FLOW

```
1. Save Gift
2. Publish GiftCreated event
```

Failure scenario:

EXAMPLE / FLOW

```
Save Gift succeeds
    ↓
Message broker fails
```

Now:

EXAMPLE / FLOW

```
Gift exists
but downstream systems never hear about it.
```

Reverse ordering is also dangerous:

EXAMPLE / FLOW

```
Publish event
    ↓
database transaction fails
```

Now downstream systems think the Gift exists when it doesn't.

Transactional Outbox

Instead:

EXAMPLE / FLOW

```
SQL TRANSACTION
    ↓
Insert/Update Gift
    +
Insert OutboxMessage
    ↓
Commit
```

Both happen together.

Then later:

EXAMPLE / FLOW

```
Outbox Processor
    ↓
```

```
read unpublished messages
        ↓
publish
```

SECURITY / PROD

Your:

EXAMPLE / FLOW

```
integration.OutboxMessage
```

fits this architecture naturally.

Outbox Entity

Conceptually:

C#

```
public sealed class OutboxMessage
{
    public Guid MessageId
    {
        get;
        init;
    }

    public string Type
    {
        get;
        init;
    } = string.Empty;

    public string Payload
    {
        get;
        init;
    } = string.Empty;

    public DateTimeOffset OccurredAt
    {
        get;
        init;
    }

    public DateTimeOffset? ProcessedAt
    {
        get;
        set;
    }
}
```

Outbox Processing Is Usually At-Least-Once

Consider:

EXAMPLE / FLOW

```
Publish message succeeds
    ↓
worker crashes
before marking ProcessedAt
```

Worker restarts:

EXAMPLE / FLOW

```
message appears unpublished
    ↓
publishes again
```

SECURITY / PROD

Therefore consumers must often tolerate duplicate delivery.

IDEMPOTENT MESSAGE CONSUMERS

Consumer Deduplication

A consumer can record:

EXAMPLE / FLOW

```
MessageId
```

in an Inbox/ProcessedMessages table.

Conceptually:

EXAMPLE / FLOW

```
Message arrives
    ↓
Has MessageId already been processed?
    ↓
YES → ignore safely

NO
    ↓
Perform work
    ↓
Record MessageId
```

This produces an **effectively-once business outcome** on top of at-least-once message delivery.

Integration Events Need Stable Contracts

Do not publish your entire EF entity:

C#

```
Publish(
    giftEntity);
```

Use explicit contracts:

C#

```
public sealed record
    GiftCompletedIntegrationEvent(
        Guid MessageId,
        long TenantId,
        long GiftId,
        decimal Amount,
        DateTimeOffset CompletedAt);
```

Now external consumers depend on:

EXAMPLE / FLOW

```
intentional integration contract
```

rather than your internal entity structure.

SECURITY / PROD

Event Versioning

A message contract is also an API.

Changing:

EXAMPLE / FLOW

```
Amount: decimal
```

to:

EXAMPLE / FLOW

```
Money:
{
    Value,
    Currency
}
```

may break old consumers.

Strategies include:

EXAMPLE / FLOW

```
additive changes

versioned events

compatibility periods

consumer-driven coordination
```

Treat integration contracts as public boundaries.

BACKGROUND SERVICES

BackgroundService

Outbox publisher:

C#

```
public sealed class
    OutboxBackgroundService(
        IServiceScopeFactory scopeFactory)
    :
        BackgroundService
{
    protected override async Task
        ExecuteAsync(
            CancellationToken stoppingToken)
    {
        while (
            !stoppingToken
                .IsCancellationRequested)
        {
            await using AsyncServiceScope scope = scopeFactory.CreateAsyncScope();

            OutboxProcessor processor =
                scope.ServiceProvider
                    .GetRequiredService<
                        OutboxProcessor>();
```

```
            await processor.ProcessAsync(
                stoppingToken);

            await Task.Delay(
                TimeSpan.FromSeconds(
                    2),
                stoppingToken);
        }
    }
}
```

SECURITY / PROD

A hosted service is singleton, so scoped dependencies such as `DbContext` should be resolved inside an explicit scope.

Propagate Shutdown Cancellation

Bad:

C#

```
await processor.ProcessAsync(
    CancellationToken.None);
```

Better:

C#

```
await processor.ProcessAsync(
    stoppingToken);
```

During deployment:

EXAMPLE / FLOW

```
Host receives shutdown
    ↓
stoppingToken cancelled
    ↓
worker stops safely
```

Background processing must cooperate with graceful shutdown.

HEALTH CHECKS

Why Health Checks Exist

Orchestrators/load balancers need to know:

EXAMPLE / FLOW

```
Is the process alive?

Is this instance ready to receive traffic?
```

ASP.NET Core provides health-check infrastructure.

Example:

C#

```
builder.Services
    .AddHealthChecks()
    .AddDbContextCheck<
        FundraisingDbContext>();
```

Liveness vs Readiness

SECURITY / PROD

Microsoft distinguishes:

EXAMPLE / FLOW

```
Liveness
    →
Is process alive,
or should it be restarted?

Readiness
    →
Is process ready to receive traffic?
```

This distinction is very important in container orchestration.

Why Database Should Not Always Be Liveness

Suppose SQL Server is temporarily unavailable.

If liveness includes SQL:

EXAMPLE / FLOW

```
SQL outage
    ↓
health fails
    ↓
orchestrator restarts API
    ↓
SQL still unavailable
    ↓
restart loop
```

The application process itself may be perfectly alive.

A database dependency is often more appropriate to:

EXAMPLE / FLOW

```
readiness
```

than:

EXAMPLE / FLOW

```
liveness
```

depending on system architecture.

Example Health Endpoints

Conceptually:

C#

```
app.MapHealthChecks(
    "/health/live",
    new HealthCheckOptions
    {
        Predicate = _ => false }).AllowAnonymous();
```

and:

```C#
app.MapHealthChecks(
    "/health/ready")
    .AllowAnonymous();
```

SECURITY / PROD

Public health responses should expose only what operations infrastructure needs.

Do not return:

EXAMPLE / FLOW

```
database password

internal server names

stack traces
```

from health endpoints.

Health Checks Are Not Monitoring

Health endpoint answers:

EXAMPLE / FLOW

```
What's the current health state?
```

Monitoring answers:

EXAMPLE / FLOW

```
How has system behaved over time?

What is latency?

What is failure rate?

Which dependency is slow?

Are errors increasing?
```

You need both.

OBSERVABILITY

Three Major Signals

Production observability commonly uses:

EXAMPLE / FLOW

```
Logs

Metrics

Traces
```

Logs

Describe discrete events:

EXAMPLE / FLOW

```
Gift completed
```

```
Payment failed
*
Message retry scheduled
```

SECURITY / PROD

Metrics

Measure values over time:

EXAMPLE / FLOW

```
Requests/sec

Error rate

P95 latency

Outbox backlog

SQL latency
```

Traces

Follow one operation across components:

EXAMPLE / FLOW

```
HTTP request
    ↓
Application
    ↓
SQL
    ↓
HTTP dependency
    ↓
Message broker
```

Useful Fundraising Metrics

Examples:

EXAMPLE / FLOW

```
HTTP request latency

HTTP 5xx rate

Payment dependency failure rate

Gift creation rate

Outbox pending count

Outbox oldest-message age

Message retry count

Database query latency

Active request count

Rate-limit rejection count
```

Business metrics and technical metrics are different but can both be important.

Alert on Symptoms That Matter

SECURITY / PROD

Bad alert:

EXAMPLE / FLOW

```
CPU > 50%
```

with no understanding of impact.

Better production questions:

EXAMPLE / FLOW

```
Is 5xx rate elevated?

Is P95 latency above objective?

Are Gifts failing to process?

Is Outbox backlog increasing?

Is payment dependency unavailable?
```

Operational alerts should help humans act.

PRODUCTION DATABASE SECURITY

Least Privilege

Your API database identity should receive only the permissions it needs.

Do not run production API as:

EXAMPLE / FLOW

```
sysadmin
```

because:

EXAMPLE / FLOW

```
"It's easier."
```

Separate:

EXAMPLE / FLOW

```
application runtime permissions
```

from:

EXAMPLE / FLOW

```
schema migration/admin permissions
```

where practical.

Application vs Migration Credentials

A production deployment may use:

EXAMPLE / FLOW

```
Migration identity
    →
```

```
DDL/schema privileges
*
Application identity
    →
normal CRUD/stored-procedure permissions
```

SECURITY / PROD

This limits blast radius if application credentials are compromised.

Backups and Recovery Are Part of Backend Design

Security and reliability include:

EXAMPLE / FLOW

```
Backup

Restore testing

Point-in-time recovery

Retention

Disaster recovery
```

A backup that has never been restored successfully is not proven recovery.

DEPLOYMENT AND SCALE

Stateless API Instances

To scale out:

EXAMPLE / FLOW

```
Load Balancer
    ↓
API #1
API #2
API #3
```

do not store critical business state only in:

EXAMPLE / FLOW

```
static dictionary

process memory

singleton field
```

because the next request may reach another instance.

Durable/shared state belongs in:

EXAMPLE / FLOW

```
Database

Distributed cache

Message broker

Durable storage
```

as appropriate.

Sticky Sessions Are Not a Substitute for Good Architecture

SECURITY / PROD

A load balancer can route a user repeatedly to one instance.

But relying on this for critical state creates problems when:

EXAMPLE / FLOW

```
instance restarts

autoscaling changes

deployment rotates instances
```

Prefer stateless APIs.

Distributed Cache

Some data can be stored in shared cache:

EXAMPLE / FLOW

```
Redis
```

for:

EXAMPLE / FLOW

```
expensive read results

short-lived distributed coordination

session state where required
```

But cache introduces:

EXAMPLE / FLOW

```
staleness

expiration

invalidation

availability

serialization
```

Cache is not automatically the source of truth.

Cache Tenant-Aware Keys

Bad:

EXAMPLE / FLOW

```
Gift:125
```

if IDs can overlap or tenant context matters.

Safer:

EXAMPLE / FLOW

```
Tenant:10:Gift:125
```

Never allow one tenant's cached data to be returned to another tenant.

SECURITY / PROD

DATA PROTECTION AND SERVER FARMS

Shared Cryptographic State

Cookie authentication and antiforgery rely on cryptographic protection.

In a multi-instance server farm, instances must be configured so protected data can be validated appropriately across instances.

Microsoft's antiforgery guidance specifically notes that Data Protection must be configured correctly for server-farm scenarios.

Otherwise:

EXAMPLE / FLOW

```
Instance A issues token/cookie
    ↓
Request goes to Instance B
    ↓
Instance B cannot validate it
```

depending on configuration.

DEPLOYMENT-SAFE DATABASE CHANGES

Avoid Breaking Rolling Deployments

Suppose:

EXAMPLE / FLOW

```
Old API instance
+
New API instance
```

run simultaneously during deployment.

A database migration that instantly removes a column still used by the old instance can break production.

Safer sequence often resembles:

EXAMPLE / FLOW

```
1. Add backward-compatible schema

2. Deploy code that supports new schema

3. Migrate/backfill data

4. Remove old usage

5. Remove old schema later
```

This is sometimes called:

EXAMPLE / FLOW

```
expand and contract
```

deployment.

Don't Let Every API Instance Run Dangerous Migrations

SECURITY / PROD

If 20 instances start simultaneously and all execute:

```
C#
Database.MigrateAsync()
```

you may create:

```
EXAMPLE / FLOW
deployment races

lock contention

migration permissions problems
```

Many production systems run schema migrations as a controlled deployment step rather than ordinary application startup.

GRACEFUL SHUTDOWN

Why Graceful Shutdown Matters

During:

```
EXAMPLE / FLOW
deployment

container restart

scale-in

host shutdown
```

an instance may be processing:

```
EXAMPLE / FLOW
HTTP request

outbox message

background job
```

Abrupt termination can produce:

```
EXAMPLE / FLOW
partial work

duplicate processing

lost progress
```

Cancellation Propagation

HTTP controller:

```
C#
public async Task<IActionResult>
    Create(
```

```
        CreateGiftRequest request,
        CancellationToken cancellationToken)
```

SECURITY / PROD

propagate through:

EXAMPLE / FLOW

```
Controller
    ↓
Application
    ↓
Repository
    ↓
EF Core
    ↓
HttpClient
```

Background service uses:

C#

```
stoppingToken
```

from the host.

Do Not Swallow OperationCanceledException Blindly

Cancellation may mean:

EXAMPLE / FLOW

```
client disconnected
```

or:

EXAMPLE / FLOW

```
application shutting down
```

Do not log every expected cancellation as:

EXAMPLE / FLOW

```
Critical system failure
```

Distinguish expected cancellation from true unexpected failures.

SECURITY HEADERS

Useful HTTP Security Headers

Depending on the application and where headers are managed, examples include:

EXAMPLE / FLOW

```
Strict-Transport-Security

X-Content-Type-Options

Content-Security-Policy

frame-ancestors via CSP

Referrer-Policy
```

A JSON-only API may not need the same CSP policy as a site serving HTML/JavaScript.

Security headers should match:

SECURITY / PROD

EXAMPLE / FLOW

```
what the application actually serves
```

and may be managed at:

EXAMPLE / FLOW

```
ASP.NET Core

reverse proxy

API gateway

CDN
```

Avoid blindly pasting every possible header.

PRODUCTION PROGRAM.CS

Example Production-Oriented Registration

Illustrative architecture:

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddControllers();

builder.Services
    .AddProblemDetails();

builder.Services
    .AddExceptionHandler<
        GlobalExceptionHandler>();

builder.Services
    .AddAuthentication(
        JwtBearerDefaults
            .AuthenticationScheme)
    .AddJwtBearer(
        options =>
        {
            options.Authority = builder.Configuration["Authentication:Authority"];

            options.Audience = builder.Configuration["Authentication:Audience"];
        });

builder.Services
    .AddAuthorization(
        options =>
        {
            options.FallbackPolicy =
                new AuthorizationPolicyBuilder()
                    .RequireAuthenticatedUser()
                    .Build();

            options.AddPolicy(
                "CanRefundGift",
                policy =>
```

```
                    policy.RequireClaim(
                        "permission",
                        "gifts.refund"));
        });

builder.Services
    .AddCors(
        options =>
        {
            options.AddPolicy(
                "Angular",
                policy =>
                    policy
                        .WithOrigins(
                            builder.Configuration[
                                "Frontend:Origin"]!)
                        .AllowAnyHeader()
                        .AllowAnyMethod());
        });

builder.Services
    .AddRateLimiter(
        options =>
        {
            // Production policies here.
        });

builder.Services
    .AddHealthChecks()
    .AddDbContextCheck<
        FundraisingDbContext>();

builder.Services
    .AddFundraisingApplication();

builder.Services
    .AddFundraisingInfrastructure(
        builder.Configuration);

builder.Services
    .AddHttpClient<
        IPaymentGateway,
        PaymentGateway>(
            client =>
            {
                client.BaseAddress = new Uri(builder.Configuration["Payment:BaseUrl"]!);
            })
    .AddStandardResilienceHandler();

var app = builder.Build();
```

SECURITY / PROD

The exact authentication/provider settings depend on the identity system.

Example Production Pipeline

C#

```
app.UseForwardedHeaders();

app.UseExceptionHandler();

if (!app.Environment.IsDevelopment())
{
    app.UseHsts();
```

```
}

app.UseHttpsRedirection();

app.UseRouting();

app.UseCors(
    "Angular");

app.UseRateLimiter();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

app.MapHealthChecks(
        "/health/live")
    .AllowAnonymous();

app.MapHealthChecks(
        "/health/ready")
    .AllowAnonymous();

app.Run();
```

SECURITY / PROD

Actual production ordering must match your:

EXAMPLE / FLOW

```
proxy

CORS

authentication

rate-limit

health-check
```

requirements.

Forwarded headers must be configured to trust only legitimate proxies.

COMPLETE FUNDRAISING SECURITY FLOW

Create Gift Request

Client:

EXAMPLE / FLOW

```
POST /api/gifts
Authorization: Bearer <token>
Content-Type: application/json
Idempotency-Key: 39fc...

{
  "constituentId": 100,
  "campaignId": 20,
  "amount": 500
}
```

Notice what's missing:

EXAMPLE / FLOW

```
TenantId
CreatedBy
Status
```

SECURITY / PROD

Those values are server-controlled.

Request Processing

EXAMPLE / FLOW

```
Kestrel
    ↓
Forwarded headers
    ↓
Exception handler
    ↓
CORS
    ↓
Rate limiter
    ↓
Authentication
    ↓
Token validated
    ↓
ClaimsPrincipal created
    ↓
Authorization
    ↓
Tenant context established
    ↓
Controller
```

Application Flow

EXAMPLE / FLOW

```
CreateGiftRequest
    ↓
Boundary validation
    ↓
CreateGiftCommand
    ↓
TenantId from trusted tenant context
    ↓
Application authorization
    ↓
Gift.Create()
    ↓
Domain validation
    ↓
Repository
    ↓
SQL transaction
```

Transactional Integration

EXAMPLE / FLOW

```
Insert Gift
+
Insert GiftAllocation
```

```
+
Insert OutboxMessage
    ↓
SaveChanges
    ↓
one database transaction
```

SECURITY / PROD

Response:

EXAMPLE / FLOW

```
201 Created
```

The email/integration doesn't need to block the request.

Background Flow

EXAMPLE / FLOW

```
Outbox Worker
    ↓
Load pending OutboxMessage
    ↓
Publish GiftCreated
    ↓
Broker
    ↓
Receipt Consumer
    ↓
Check MessageId deduplication
    ↓
Generate receipt
    ↓
Send external email
```

If email fails:

EXAMPLE / FLOW

```
retry background integration
```

rather than:

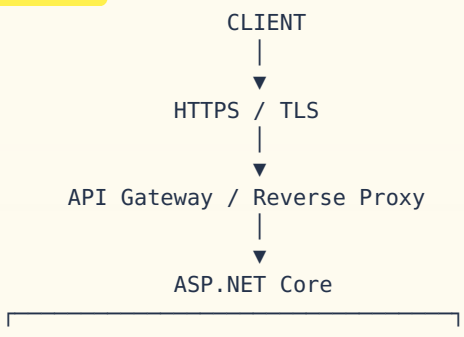
EXAMPLE / FLOW

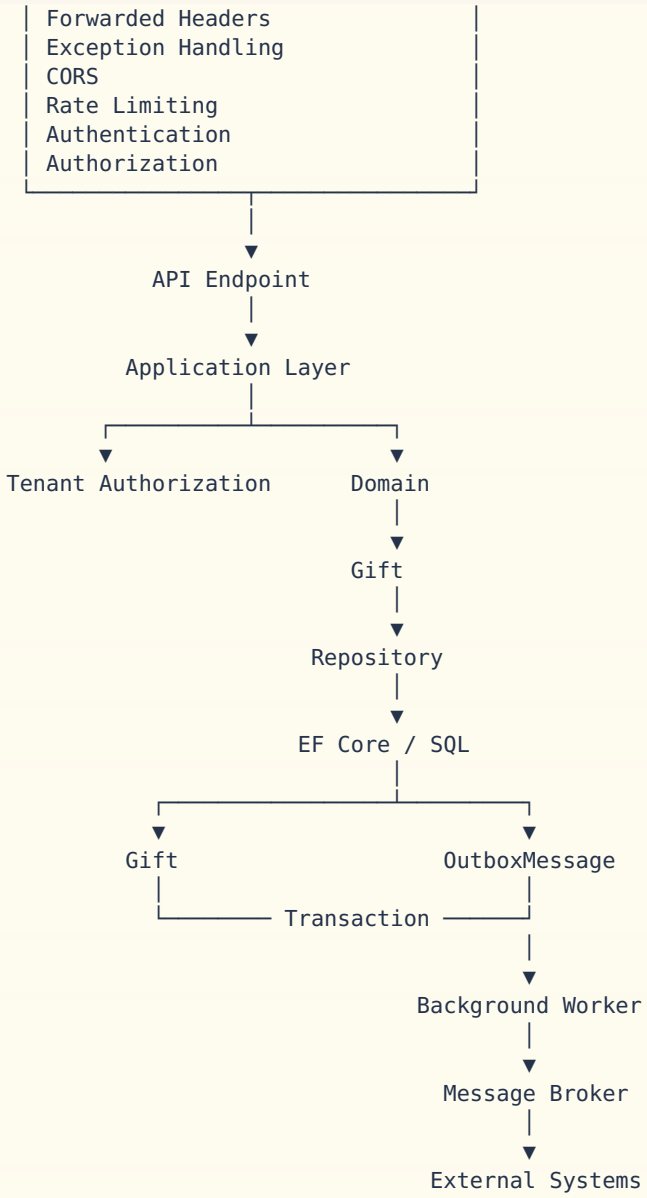
```
rollback already-created Gift
```

unless business rules explicitly require otherwise.

Diagram 1 — Production Fundraising Backend

EXAMPLE / FLOW





SECURITY / PROD

FAILURE DESIGN

SQL Server Is Down

Request:

EXAMPLE / FLOW

```
POST /api/gifts
```

SQL Server unavailable.

Correct behavior:

EXAMPLE / FLOW

```
request fails

no fake success response

exception logged

safe 5xx response
```

```
health readiness may fail
```

SECURITY / PROD

Do not return:

EXAMPLE / FLOW

```
201 Created
```

before durable persistence succeeds.

Payment Provider Is Slow

Use:

EXAMPLE / FLOW

```
timeout

resilience strategy

circuit breaker

bounded concurrency
```

Do not allow:

EXAMPLE / FLOW

```
every request waits indefinitely
```

Message Broker Is Down

With Outbox:

EXAMPLE / FLOW

```
Gift transaction can succeed
    ↓
OutboxMessage remains pending
    ↓
Publisher retries later
```

This isolates database success from temporary broker availability.

Duplicate Message Arrives

Consumer:

EXAMPLE / FLOW

```
checks MessageId
```

If already processed:

EXAMPLE / FLOW

```
return success / ignore
```

without repeating the business effect.

API Instance Crashes

SECURITY / PROD

Because important state is in:

EXAMPLE / FLOW

```
SQL

broker

durable stores
```

another instance can continue.

Critical business data must not exist only in:

EXAMPLE / FLOW

```
process memory
```

PERFORMANCE AND SECURITY TOGETHER

Many Security Controls Also Protect Performance

Examples:

EXAMPLE / FLOW

```
Rate limiting
    →
prevents excessive request load

Pagination
    →
prevents huge queries

Request-size limits
    →
prevents giant payloads

Timeouts
    →
prevent resource starvation

Concurrency limits
    →
protect dependencies

Authorization
    →
prevents unauthorized expensive operations
```

Security and reliability often reinforce one another.

Avoid Expensive Authorization N+1

Bad:

EXAMPLE / FLOW

```
Get 500 Gifts
    ↓
for each Gift
```

```
    ↓
authorization query to database
```

SECURITY / PROD

This can produce:

EXAMPLE / FLOW

```
500 extra queries
```

Design permissions and queries together.

For example, query only Gifts the current Tenant/user is permitted to access when possible.

Avoid Security Through Obscurity

Bad assumption:

EXAMPLE / FLOW

```
Gift IDs are GUIDs,
so users cannot guess them.
```

Even unguessable IDs do not replace authorization.

Always verify access.

SECURITY TESTING

Integration Tests Should Include Security

Important API tests:

EXAMPLE / FLOW

```
No token
    →
401

Valid user without permission
    →
403

Tenant A requests Tenant B Gift
    →
404/403 according to contract

Invalid token
    →
401

Allowed user
    →
success
```

Test Tenant Isolation

Arrange:

EXAMPLE / FLOW

```
Tenant A
    Gift 100
```

```
Tenant B
 *  Gift 200
```

SECURITY / PROD

Authenticate as Tenant A.

Verify:

EXAMPLE / FLOW

```
GET /api/gifts/200
```

does not expose Tenant B data.

This is one of the most important tests in a multi-tenant application.

Test Idempotency

Send:

EXAMPLE / FLOW

```
same Idempotency-Key
```

twice.

Verify:

EXAMPLE / FLOW

```
one Gift
```

rather than:

EXAMPLE / FLOW

```
two Gifts.
```

Test Rate Limiting

Microsoft specifically recommends load testing and reviewing rate-limiting policies before deployment.

Test:

EXAMPLE / FLOW

```
allowed request count

rejection behavior

partition isolation

burst behavior

queue behavior
```

Test Integration Failure

Simulate:

EXAMPLE / FLOW

```
Payment API timeout

Payment API 500

Broker unavailable
```

```
Duplicate broker message

SQL deadlock/transient error
```

SECURITY / PROD

Verify your intended resilience semantics.

COMMON PRODUCTION MISTAKES

Mistake 1 — Authentication Without Authorization

EXAMPLE / FLOW

```
Token valid
    →
therefore allow everything
```

Wrong.

Authentication proves identity, not permissions.

Mistake 2 — Trusting TenantId from Client

Client input must not control security scope.

Mistake 3 — CORS as Security

EXAMPLE / FLOW

```
Only Angular origin allowed
    →
API is secure
```

False.

Non-browser clients can call it directly.

Mistake 4 — AllowAnyOrigin + Credentials

Microsoft explicitly warns this is unsafe.

Mistake 5 — Secrets in appsettings.json

Especially dangerous when committed to Git.

Mistake 6 — User Secrets in Production

User Secrets are not encrypted and are meant for development.

Mistake 7 — New HttpClient Per Request

Can lead to poor connection management.

Use:

EXAMPLE / FLOW

```
IHttpClientFactory
```

or a correctly managed long-lived `HttpClient` / `SocketsHttpHandler` strategy.

Mistake 8 — Retry Every Failure

Retrying a non-idempotent payment can duplicate charges.

Mistake 9 — No Timeouts

A slow dependency can consume your entire request capacity.

Mistake 10 — Database + Message Broker Dual Write

Use a reliable pattern such as the Outbox when both results must be coordinated.

SECURITY / PROD

Mistake 11 — No Consumer Idempotency

At-least-once delivery can create duplicate business effects.

Mistake 12 — Liveness Depends on Every External System

A temporary dependency failure can cause restart loops.

Mistake 13 — Logging Tokens and Secrets

Logs are persistent data stores.

Treat them accordingly.

Mistake 14 — Unbounded Queries

EXAMPLE / FLOW

```
GET /gifts
```

returning millions of rows is both reliability and security/resource-abuse risk.

Mistake 15 — Shared Singleton Tenant State

C#

```
public long CurrentTenantId;
```

on a singleton can leak one request's tenant into another.

Mistake 16 — Trusting X-Forwarded-For from Everyone

Forwarded headers must come from trusted proxies.

Mistake 17 — Auto-Migrating From Every Instance

Production migrations need controlled deployment design.

Mistake 18 — Exposing Detailed Health Checks Publicly

Health endpoints should not leak internal topology or secrets.

Mistake 19 — Returning Stack Traces

Production clients should receive safe errors.

Mistake 20 — No Graceful Shutdown

Abrupt worker/request termination can create duplicate or incomplete processing.

BEST PRACTICES

For a production ASP.NET Core backend:

EXAMPLE / FLOW

```
Require HTTPS.

Use established authentication middleware.

Require authentication by default where appropriate.

Use policy-based authorization for business permissions.

Use resource authorization when permission depends on loaded data.
```

Treat tenant isolation as a first-class security requirement.

Never trust TenantId supplied by the client.

SECURITY / PROD

Use CORS only for intended browser origins.

Remember that CORS isn't authentication.

Protect cookie-authenticated browser writes from CSRF.

Store secrets outside source control.

Use production secret-management infrastructure.

Apply rate limits to abuse-prone/expensive endpoints.

Validate request size and shape.

Keep domain validation inside the Domain.

Use parameterized SQL.

Use purpose-specific DTOs.

Avoid leaking sensitive error details.

Use structured logs without secrets.

Use IHttpClientFactory or an equivalent correct HttpClient lifetime design.

Apply timeouts to external dependencies.

Retry only safely repeatable operations.

Use circuit breakers/concurrency controls for unhealthy dependencies.

Use Outbox for reliable transactional integration.

Design consumers for duplicate delivery.

Separate liveness from readiness.

Use logs, metrics, and traces.

Keep API instances stateless where possible.

Propagate CancellationToken.

Support graceful shutdown.

Use controlled database migrations.

Test authentication, authorization, tenant isolation, retries, idempotency, and production failure paths.

12 PRACTICAL EXERCISES

Exercise 1 — Authentication

Configure JWT bearer authentication.

Verify:

EXAMPLE / FLOW

```
No token
    →
401

Invalid token
    →
401

Valid token
    →
endpoint reached
```

SECURITY / PROD

Exercise 2 — Fallback Authorization

Configure:

C#

```
FallbackPolicy
```

to require an authenticated user.

Create one:

C#

```
[AllowAnonymous]
```

health endpoint.

Verify every other new endpoint is protected automatically.

Exercise 3 — Permission Policy

Create:

EXAMPLE / FLOW

```
CanRefundGift
```

requiring:

EXAMPLE / FLOW

```
permission = gifts.refund
```

Verify users without the claim receive:

EXAMPLE / FLOW

```
403
```

Exercise 4 — Tenant Isolation

Create:

EXAMPLE / FLOW

```
Tenant A Gift 100

Tenant B Gift 200
```

Authenticate as Tenant A.

Verify Tenant A cannot retrieve or modify Gift 200.

Exercise 5 — CORS

SECURITY / PROD

Allow:

EXAMPLE / FLOW

```
https://localhost:4200
```

or your local Angular origin.

Verify another browser origin is rejected by CORS.

Then explain why:

EXAMPLE / FLOW

```
curl
```

can still call the API.

Exercise 6 — Secrets

Move:

EXAMPLE / FLOW

```
Payment:ApiKey
```

out of `appsettings.json`.

Use User Secrets locally.

Design a production secret-source strategy.

Exercise 7 — Rate Limiting

Apply:

EXAMPLE / FLOW

```
10 Gift-create requests/minute
```

per authenticated user.

Verify one user's limit doesn't consume another user's partition.

Exercise 8 — SQL Injection Review

Find every use of:

EXAMPLE / FLOW

```
FromSqlRaw

ExecuteSqlRaw

string-built SQL
```

and verify that untrusted values cannot alter SQL syntax.

Exercise 9 — Resilient HttpClient

Create:

C#

```
IPaymentGateway
```

using a typed `HttpClient`.

Add:

C#

```
AddStandardResilienceHandler()
```

SECURITY / PROD

Then test:

EXAMPLE / FLOW

```
timeout

500

temporary failure
```

and decide which requests are safe to retry.

Exercise 10 — Transactional Outbox

During Gift completion:

EXAMPLE / FLOW

```
Update Gift

Insert OutboxMessage

SaveChanges
```

in one transaction.

Simulate message-broker outage and verify the Gift remains committed while Outbox remains pending.

Exercise 11 — Idempotent Consumer

Send the same:

EXAMPLE / FLOW

```
GiftCompletedIntegrationEvent
```

twice with the same:

EXAMPLE / FLOW

```
MessageId
```

Verify only one receipt is generated.

Exercise 12 — Production Health

Create:

EXAMPLE / FLOW

```
/health/live

/health/ready
```

Make SQL Server failure affect readiness without unnecessarily causing the liveness probe to report the process as dead.

EXACTLY 25 INTERVIEW QUESTIONS

1. What is the difference between authentication and authorization?

Authentication determines:

EXAMPLE / FLOW

```
Who is the caller?
```

SECURITY / PROD

Authorization determines:

EXAMPLE / FLOW

```
What is that caller allowed to do?
```

ASP.NET Core authentication builds a `ClaimsPrincipal` from registered authentication schemes/handlers. Authorization then evaluates the identity against roles, claims, policies, requirements, or resources. Authentication alone does not restrict endpoints.

2. What is an authentication scheme in ASP.NET Core?

An authentication scheme is a named authentication configuration backed by an authentication handler.

Examples include:

EXAMPLE / FLOW

```
JWT Bearer

Cookies
```

Registration might look like:

C#

```
builder.Services
    .AddAuthentication(
        JwtBearerDefaults
            .AuthenticationScheme)
    .AddJwtBearer();
```

The handler validates credentials and creates the authenticated identity.

3. Does AddAuthentication automatically secure every endpoint?

No.

Authentication configuration establishes how identities can be authenticated.

Authorization determines which endpoints actually require authentication.

A strong secure-by-default server configuration can use a fallback authorization policy requiring authenticated users, then mark intentionally public endpoints with `[AllowAnonymous]`.

4. What is policy-based authorization?

A policy is one or more authorization requirements.

For example:

EXAMPLE / FLOW

```
CanRefundGift
    ↓
authenticated
+
permission = gifts.refund
```

ASP.NET Core authorization handlers evaluate requirements and determine whether the policy succeeds.

5. What is resource-based authorization?

SECURITY / PROD

Resource-based authorization evaluates permission using the actual resource being accessed.

Example:

EXAMPLE / FLOW

```
User may edit Gift
only when Gift belongs to an allowed Tenant.
```

Because the resource usually must be loaded before the decision can be made, ASP.NET Core supports imperative authorization through:

C#

```
IAuthorizationService.AuthorizeAsync(...)
```

rather than relying only on `[Authorize]` metadata.

6. How would you protect multi-tenant data?

Use defense in depth:

EXAMPLE / FLOW

```
Validated authenticated identity

Trusted Tenant context

Authorization policies

Tenant-aware repositories/queries

Database constraints

Security integration tests
```

Never trust a request-body TenantId simply because the client supplied it.

Every data-access operation involving tenant-owned resources should preserve the intended Tenant boundary.

7. What is CORS?

CORS is a browser mechanism allowing a server to relax the browser's same-origin policy for specified origins.

It controls which cross-origin responses browser JavaScript can access.

It is not authentication or authorization, and non-browser HTTP clients don't rely on CORS enforcement. Microsoft explicitly describes CORS as relaxing security rather than providing authentication/security itself.

8. Why is AllowAnyOrigin with credentials dangerous?

Credentials such as authentication cookies may be automatically attached by the browser.

Allowing arbitrary origins to make credentialed requests can allow untrusted websites to interact with authenticated sessions.

ASP.NET Core rejects the insecure combination of:

EXAMPLE / FLOW

```
AllowAnyOrigin
+
AllowCredentials
```

SECURITY / PROD

and Microsoft explicitly warns against it.

9. What is CSRF?

CSRF is an attack where a malicious website causes a victim's browser to make a state-changing request to another site using automatically included credentials.

It is especially relevant to:

EXAMPLE / FLOW

```
cookie authentication

Windows authentication
```

ASP.NET Core provides antiforgery protection for appropriate browser/form scenarios.

10. Why is HTTPS required even if the API uses JWTs?

Without TLS, an attacker observing the network may steal:

EXAMPLE / FLOW

```
Bearer token

request content

personal data

responses
```

JWT authentication protects identity verification; it doesn't encrypt the network transport.

HTTPS protects data in transit.

Microsoft recommends HTTPS/TLS for production ASP.NET Core applications.

11. What are forwarded headers and why do they matter?

When ASP.NET Core is behind a reverse proxy, Kestrel may see:

EXAMPLE / FLOW

```
proxy IP

HTTP scheme
```

instead of:

EXAMPLE / FLOW

```
real client IP

original HTTPS scheme
```

Forwarded headers such as `X-Forwarded-For` and `X-Forwarded-Proto` provide that original information.

ASP.NET Core's middleware can apply those values to `HttpContext`, but only trusted proxies should be allowed to supply them.

12. How should application secrets be stored?

SECURITY / PROD

Do not put production secrets in:

EXAMPLE / FLOW

```
source code

Git

appsettings.json
```

For local development, User Secrets can keep values outside the project tree, but Microsoft notes that User Secrets aren't encrypted and are intended only for development.

Production secrets should come from a controlled secret-management mechanism such as a cloud vault.

13. What is ASP.NET Core rate limiting?

Rate limiting restricts incoming traffic based on configured policies.

ASP.NET Core provides:

EXAMPLE / FLOW

```
Fixed window

Sliding window

Token bucket

Concurrency limiting
```

Policies may be partitioned by user, tenant, API key, IP, or another identifier.

For the Fundraising Platform, expensive write/export endpoints are good candidates for deliberate rate limits.

14. How does EF Core help prevent SQL injection?

Normal LINQ queries send user values as parameters rather than concatenating them into SQL syntax.

EF Core raw-SQL APIs also provide parameterized approaches.

Danger increases when application code dynamically inserts raw untrusted strings into SQL, especially schema components such as column names that cannot be parameterized.

Use allow-lists and parameterization.

15. What is IHttpClientFactory and why use it?

`IHttpClientFactory` provides centralized configuration and creation of `HttpClient` instances.

It manages underlying handler pooling/lifetimes and supports:

EXAMPLE / FLOW

```
named clients

typed clients

logging

delegating handlers

resilience pipelines
```

This helps avoid common connection-lifetime and DNS problems associated with incorrect `HttpClient` creation patterns.

SECURITY / PROD

16. What is the danger of retrying every HTTP failure?

A retry can repeat a business operation.

Example:

EXAMPLE / FLOW

```
Charge card
    ↓
provider succeeds
    ↓
network response lost
    ↓
retry
    ↓
second charge
```

Retries should be used only when the operation is safely repeatable or protected by an idempotency mechanism.

Transient-fault handling must understand HTTP/business semantics.

17. What is a circuit breaker?

A circuit breaker temporarily stops requests to a dependency after repeated failures.

Conceptually:

EXAMPLE / FLOW

```
Closed
    →
normal calls

Open
    →
calls blocked quickly

Half-open
    →
trial request
```

This prevents every incoming request from repeatedly waiting on a dependency known to be unhealthy.

Modern .NET resilience libraries support circuit-breaker strategies.

18. What is AddStandardResilienceHandler?

`AddStandardResilienceHandler()` adds a standard resilience pipeline to a configured `HttpClient`.

The current .NET pipeline combines strategies such as:

EXAMPLE / FLOW

```
request/concurrency limiting

total timeout

retry

circuit breaker
```

```
attempt timeout
```

SECURITY / PROD

and can be customized for a dependency's requirements.

19. What problem does the Outbox pattern solve?

It addresses the dual-write problem:

EXAMPLE / FLOW

```
database update
+
message publication
```

Instead of trying to make two independent systems succeed simultaneously, store:

EXAMPLE / FLOW

```
business change
+
OutboxMessage
```

in one database transaction.

A background worker publishes the message later.

This prevents committed business state from silently losing the integration message.

20. Why must message consumers often be idempotent?

Many durable messaging systems provide at-least-once delivery semantics.

A consumer may therefore receive:

EXAMPLE / FLOW

```
Message A

Message A again
```

If processing creates a receipt or payment twice, the result is incorrect.

Track a unique `MessageId` or business idempotency key so duplicate delivery produces one business effect.

21. What is the difference between liveness and readiness?

Liveness asks:

EXAMPLE / FLOW

```
Is the process alive,
or should it be restarted?
```

Readiness asks:

EXAMPLE / FLOW

```
Is this instance currently ready
to receive application traffic?
```

Microsoft recommends treating these as distinct health states in appropriate hosting environments.

For example, SQL outage may make an API not ready without necessarily meaning its process should be restarted.

22. What is the difference between health checks and observability?

SECURITY / PROD

Health checks provide a current operational state:

EXAMPLE / FLOW

```
Healthy

Degraded

Unhealthy
```

Observability explains system behavior over time using:

EXAMPLE / FLOW

```
Logs

Metrics

Traces
```

A health check may tell you the database is unhealthy.

Tracing/metrics/logging help explain:

EXAMPLE / FLOW

```
when it began

how many requests were affected

which dependency failed

what latency changed
```

23. Why should ASP.NET Core APIs be stateless when scaling horizontally?

With multiple instances:

EXAMPLE / FLOW

```
Request 1
    →
API instance A

Request 2
    →
API instance B
```

If important business/session state exists only in instance A's memory, request 2 cannot see it.

Durable/shared state should instead live in:

EXAMPLE / FLOW

```
database

distributed cache

broker

durable storage
```

where appropriate.

Stateless application instances scale and recover more predictably.

24. What is graceful shutdown and why is CancellationToken important?

SECURITY / PROD

During deployment or process termination, the host signals cancellation.

HTTP/application/database/external calls should propagate cancellation tokens so unnecessary work can stop.

Background services receive a:

```
C#
stoppingToken
```

and should stop accepting/starting new work when shutdown begins.

Graceful shutdown reduces:

```
EXAMPLE / FLOW
partial processing

lost work

resource leaks
```

and combines well with idempotent message processing.

25. Describe a production-ready Fundraising Platform backend.

A strong architecture could look like:

```
EXAMPLE / FLOW
Client
    ↓
HTTPS
    ↓
Gateway / Reverse Proxy
    ↓
ASP.NET Core
    ↓
Forwarded Headers
    ↓
Exception Handling
    ↓
CORS
    ↓
Rate Limiting
    ↓
Authentication
    ↓
Authorization
    ↓
Tenant Context
    ↓
Controller / Endpoint
    ↓
Application Layer
    ↓
Domain
    ↓
Repository / EF Core
    ↓
SQL Server
```

For writes:

SECURITY / PROD

EXAMPLE / FLOW

```
Gift
+
OutboxMessage
    ↓
one database transaction
```

Then:

EXAMPLE / FLOW

```
Background Outbox Worker
    ↓
Message Broker
    ↓
Idempotent Consumer
```

External HTTP integrations use:

EXAMPLE / FLOW

```
IHttpClientFactory

timeouts

safe retry semantics

circuit breaker

bounded concurrency
```

Operationally, the system includes:

EXAMPLE / FLOW

```
secret management

liveness/readiness probes

structured logging

metrics

distributed tracing

rate limits

tenant-isolation tests

controlled migrations

backups

graceful shutdown
```

Security is implemented at multiple boundaries rather than by one `[Authorize]` attribute.

QUICK REVISION

Concept	Meaning
Authentication	Establish caller identity

Concept	Meaning
Authorization	Decide what caller may do
Authentication Scheme	Named authentication handler/configuration
JWT Bearer	Token sent in Authorization header
Claims	Identity/permission facts
Role	Named permission grouping
Policy	Collection of authorization requirements
Resource Authorization	Authorization using actual resource
Tenant Isolation	Prevent cross-tenant data access
CORS	Browser cross-origin policy relaxation
CSRF	Cross-site request using ambient credentials
HTTPS	Encryption/authentication of transport
HSTS	Browser HTTPS enforcement instruction
Forwarded Headers	Preserve original proxy request information
Secret Manager	Development secret storage
Rate Limiting	Limit request rate/concurrency
SQL Injection	Untrusted input becoming SQL syntax
`IHttpClientFactory`	Managed outbound HTTP clients
Retry	Repeat transiently failed operation
Circuit Breaker	Temporarily stop unhealthy dependency calls
Timeout	Bound operation duration
Outbox	Reliable DB → messaging bridge
Idempotency	Duplicate request/message has one intended effect
Liveness	Should process be restarted?
Readiness	Should instance receive traffic?
Logs	Discrete events
Metrics	Numerical behavior over time
Traces	Distributed operation flow
Stateless API	Durable state outside process
Graceful Shutdown	Controlled termination

SECURITY / PROD

FINAL ROADMAP CONCEPT FLOW

Everything from Topics 1–40 now connects:

EXAMPLE / FLOW

```
C# LANGUAGE
    ↓
```

```
Types
Classes
Interfaces
Generics
Collections
Delegates
LINQ
Exceptions
Async
Concurrency
Memory
    ↓
OOP + SOLID
    ↓
Dependency Injection
    ↓
Design Patterns
    ↓
Application Architecture
    ↓
Repository / Unit of Work / Services
    ↓
Testing
    ↓
ASP.NET Core Startup
    ↓
Request Pipeline
    ↓
Web APIs / REST
    ↓
Security + Integrations + Production
```

SECURITY / PROD

For one Fundraising request:

EXAMPLE / FLOW

```
ANGULAR
   ↓
HTTPS
   ↓
ASP.NET Core
   ↓
Authentication
   ↓
Authorization
   ↓
Tenant Isolation
   ↓
Model Binding
   ↓
Validation
   ↓
Controller
   ↓
Application Service
   ↓
Domain Entity
   ↓
Repository
   ↓
EF Core
   ↓
SQL Server
```

For reliable integration:

EXAMPLE / FLOW

```
Gift state change
    +
OutboxMessage
    ↓
Database transaction
    ↓
Commit
    ↓
Background worker
    ↓
Message broker
    ↓
Idempotent consumer
    ↓
External side effect
```

SECURITY / PROD

For external synchronous integration:

EXAMPLE / FLOW

```
Application
    ↓
Integration abstraction
    ↓
Typed HttpClient
    ↓
Timeout
    ↓
Retry if safe
    ↓
Circuit breaker
    ↓
External service
```

For operations:

EXAMPLE / FLOW

```
Running instance
    ↓
Liveness + Readiness

Application activity
    ↓
Logs + Metrics + Traces

Deployment
    ↓
Graceful shutdown

Scale
    ↓
Multiple stateless instances
```

For security:

EXAMPLE / FLOW

```
TLS
    ↓
Authentication
    ↓
Authorization
    ↓
```

```
Tenant isolation
    ↓
Validation
    ↓
Domain invariants
    ↓
Safe persistence
    ↓
Safe integration
    ↓
Secure secrets
    ↓
Auditable operations
```

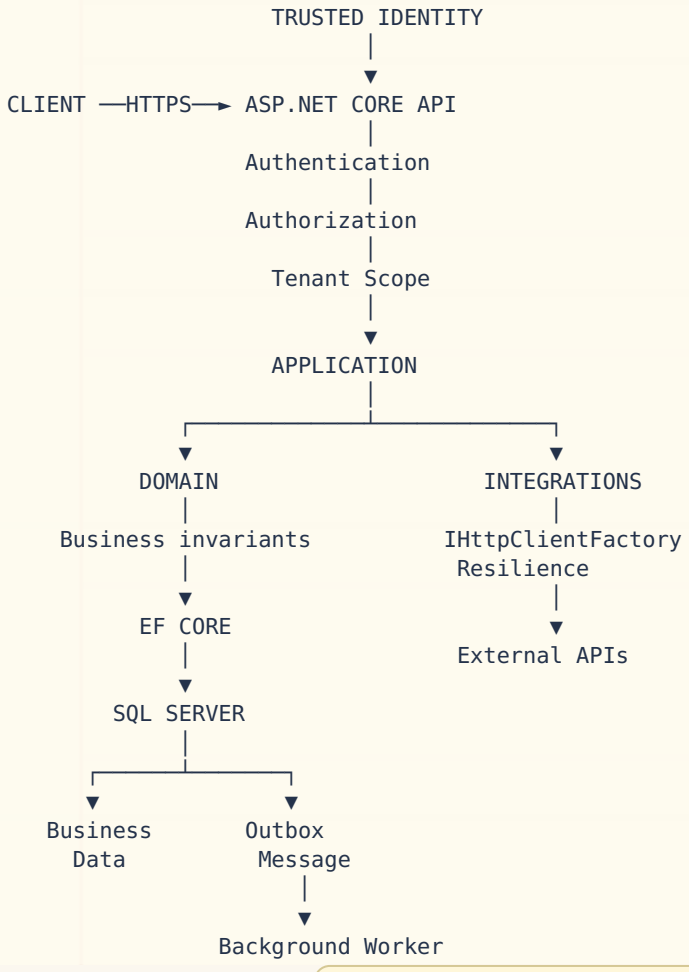
SECURITY / PROD

The final principle for the entire roadmap is:

> **A production .NET backend is not simply a collection of controllers and database queries. It is a system of carefully controlled boundaries. C# gives you the language mechanisms, OOP and SOLID structure responsibilities, DI connects components, architecture controls dependency direction, tests protect behavior, ASP.NET Core exposes the application over HTTP, authentication and authorization protect access, tenant isolation protects data ownership, resilient integration isolates external failures, transactional messaging protects consistency, and production observability and lifecycle management make the system operable after deployment.**

Final Fundraising Platform Mental Model

EXAMPLE / FLOW



```
        |
        ▼
Message Broker
        |
        ▼
Idempotent Consumer
```

SECURITY / PROD

ROADMAP COMPLETE — 40 / 40

You have now covered the complete C#/.NET backend roadmap from:

EXAMPLE / FLOW

```
Topic 1
C# and .NET Fundamentals
```

through:

EXAMPLE / FLOW

```
Topic 40
ASP.NET Core Security,
Integration and Production Backend Design
```

The natural next stage is no longer another isolated language topic.

The next stage is to **build the Fundraising Platform end-to-end** and apply Topics 1–40 together.